Álvaro Puebla Ruisánchez / Óscar López García

CYBERSECURITY REPORT – LABORATORY 7

1 Commands used:

1.1 OpenSSL:

- Use the OpenSSL tool to generate RSA private key (key length 2048 bits):
  - openssl genrsa -out private_key.pem 2048
- Based on private key, generate public.key:
  - openssl rsa -in private_key.pem -outform PEM -pubout -out public_key.pem
- Sign the file with the following commands:
  - openssl dgst -sha256 -sign private_key.pem -out encrypted_hash.sha256 data.txt
- To verify the signature, run the following command:
  - openssl dgst -sha256 -verify public_key.pem -signature encrypted_hash data.txt
- OpenSSL documentation is available here: https://www.openssl.org/docs/

1.2 OpenPGP:

- Generate key pair using GPG tool:
  - gpg --expert --full-gen-key
- Before generating the key you must complete the fields (key length, elliptic curve, …):
- Some useful commands are:
  - List public keys: > gpg --list-keys
  - List private keys: > gpg --list-secret-keys
  - Export a public key:
    - ASCII > gpg --output <file name> --export --armour <email_address/key ID>
    - Binary > gpg --export --output <file name> <email_address/key ID>
  - Export a private key: > gpg --export-secret-keys -a <file name> <email address>
  - Importuj klucz: > gpg –-import <file name>
  - Sign a file: > gpg --sign <file name>
  - Sign a file with a selected key: > gpg --sign -u <key_id> <file name>
  - Verify a signature: > gpg --verify <file name>.gpg
  - Sign an imported key: > gpg --sign-key
  - Encrypt a file with gpg: > gpg --encrypt –-recipient <email address/key ID> --output <output file> <input file>
  - Other useful operations of gpg:
    - > gpg –-fingerprint
    - > gpg –—edit-key

2 Application of modern cryptography:

2.1 Generate a set of keys using OpenSSL (RSA) and OpenPGP (ECC and RSA):

OpenSSL (RSA): To generate the private key: > openssl genrsa -out A_OpenSSL_private_key.pem 2048

```
┌──(stud㉿kali-vm)-[~]
└─$ openssl genrsa -out A_OpenSSL_private_key.pem 2048
```

To get the public key: > openssl rsa -in A_OpenSSL_private_key.pem -outform PEM -pubout -out A_OpenSSL_public_key.pem

```
┌──(stud㉿kali-vm)-[~]
└─$ openssl rsa -in A_OpenSSL_private_key.pem -outform PEM -pubout -out A_OpenSSL_public_key.pem
writing RSA key
```

OpenPGP (ECC): > gpg --expert --full-gen-key

```
┌──(stud㉿kali-vm)-[~]
└─$ gpg --expert --full-gen-key
gpg (GnuPG) 2.2.40; Copyright (C) 2022 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Please select what kind of key you want:
   (1) RSA and RSA (default)
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (7) DSA (set your own capabilities)
   (8) RSA (set your own capabilities)
   (9) ECC and ECC
  (10) ECC (sign only)
  (11) ECC (set your own capabilities)
  (13) Existing key
  (14) Existing key from card
Your selection? 9
Please select which elliptic curve you want:
   (1) Curve 25519
   (3) NIST P-256
   (4) NIST P-384
   (5) NIST P-521
   (6) Brainpool P-256
   (7) Brainpool P-384
   (8) Brainpool P-512
   (9) secp256k1
Your selection? 9
```

```
public and secret key created and signed.

pub   secp256k1 2025-11-16 [SC]
      455C3D3DFC34617220468280B1633EFDFF7A27D1
uid                      A_Kali (A_Kali_ECC_key) <AKali@gmail.com>
sub   secp256k1 2025-11-16 [E]
```

" A_Kali (A_Kali_ECC_key) <AKali@gmail.com>" -> Password: 123456789

CYBERSECURITY REPORT – LABORATORY 7

OpenPGP (RSA): > gpg --expert --full-gen-key

```
┌──(stud㉿kali-vm)-[~]
└─$ gpg --expert --full-gen-key
gpg (GnuPG) 2.2.40; Copyright (C) 2022 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Please select what kind of key you want:
   (1) RSA and RSA (default)
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (7) DSA (set your own capabilities)
   (8) RSA (set your own capabilities)
   (9) ECC and ECC
  (10) ECC (sign only)
  (11) ECC (set your own capabilities)
  (13) Existing key
  (14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072)
Requested keysize is 3072 bits
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want for the subkey? (3072)
Requested keysize is 3072 bits
```

```
public and secret key created and signed.

pub   rsa3072 2025-11-16 [SC]
      CA314C05F18ED7810FECEDC0CFD9223CA65852C3
uid                      A_Kali (A_Kali_RSA_key) <AKali@gmail.com>
sub   rsa3072 2025-11-16 [E]
```

" A_Kali (A_Kali_RSA_key) <AKali@gmail.com>" -> Password: 123456789

List the keys generated: gpg --list-keys

```
┌──(stud㉿kali-vm)-[~]
└─$ gpg --list-keys
gpg: checking the trustdb
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: depth: 0  valid:   2  signed:   0  trust: 0-, 0q, 0n, 0m, 0f, 2u
/home/stud/.gnupg/pubring.kbx
------------------------------
pub   secp256k1 2025-11-16 [SC]
      455C3D3DFC34617220468280B1633EFDFF7A27D1
uid           [ultimate] A_Kali (A_Kali_ECC_key) <AKali@gmail.com>
sub   secp256k1 2025-11-16 [E]

pub   rsa3072 2025-11-16 [SC]
      CA314C05F18ED7810FECEDC0CFD9223CA65852C3
uid           [ultimate] A_Kali (A_Kali_RSA_key) <AKali@gmail.com>
sub   rsa3072 2025-11-16 [E]
```

2.2 Export a public key from GPG (ACSII format):

By > gpg --output A_Kali_ECC_public_key --export --armour 455C3D3DFC34617220468280B1633EFDFF7A27D1

and gpg --output A_Kali_RSA_public_key --export --armour CA314C05F18ED7810FECEDC0CFD9223CA65852C3:

```
┌──(stud㉿kali-vm)-[~]
└─$ gpg --output A_Kali_ECC_public_key --export --armour 455C3D3DFC34617220468280B1633EFDFF7A27D1

┌──(stud㉿kali-vm)-[~]
└─$ gpg --output A_Kali_RSA_public_key --export --armour CA314C05F18ED7810FECEDC0CFD9223CA65852C3
```

2.3 Transfer your public key (OpenSSL and OpenPGP) to another machine/user account:

Transferring the files as: sftp student@192.168.144.5 (Where 192.168.144.5 is the ubuntu machine: **ip a**)

```
┌──(stud㉿kali-vm)-[~]
└─$ sftp student@192.168.144.5
student@192.168.144.5's password:
Connected to 192.168.144.5.
sftp> put A_OpenSSL_public_key.pem
Uploading A_OpenSSL_public_key.pem to /home/student/A_OpenSSL_public_key.pem
A_OpenSSL_public_key.pem                                    100%  451   130.0KB/s   00:00
sftp> put A_Kali_ECC_public_key
Uploading A_Kali_ECC_public_key to /home/student/A_Kali_ECC_public_key
A_Kali_ECC_public_key                                       100%  725   430.5KB/s   00:00
sftp> put A_Kali_RSA_public_key
Uploading A_Kali_RSA_public_key to /home/student/A_Kali_RSA_public_key
A_Kali_RSA_public_key                                       100% 2448   280.7KB/s   00:00
sftp> quit
```

Transferred the files: put A_OpenSSL_public_key.pem / put A_Kali_ECC_public_key / put A_Kali_RSA_public_key

```
student@student-ubuntu:~$ ls
A_Kali_ECC_public_key  A_OpenSSL_public_key.pem
A_Kali_RSA_public_key  Desktop
```

So now all the public keys are in the ubuntu machine.

2.4 For OpenPGP, a public key should be imported and signed with a local private key. Verify a fingerprint of a certificate on both machines/ user accounts:

We label the machines **-> A** (key owner - Kali) and **B** (signer - Ubuntu):

B imports A's public keys: > gpg --import A_Kali_ECC_public_key

and > gpg –import A_Kali_RSA_public_key

```
student@student-ubuntu:~$ gpg --import A_Kali_ECC_public_key
gpg: directory '/home/student/.gnupg' created
gpg: keybox '/home/student/.gnupg/pubring.kbx' created
gpg: /home/student/.gnupg/trustdb.gpg: trustdb created
gpg: key B1633EFDFF7A27D1: public key "A_Kali (A_Kali_ECC_key) <AKali@gmail.com>" imported
gpg: Total number processed: 1
gpg:               imported: 1
student@student-ubuntu:~$ gpg --import A_Kali_RSA_public_key
gpg: key CFD9223CA65852C3: public key "A_Kali (A_Kali_RSA_key) <AKali@gmail.com>" imported
gpg: Total number processed: 1
gpg:               imported: 1
```

The fingerprint is calculated by hashing 0x99 | 2 bytes of the public key length | public key.

Check **A's fingerprints** and **B's fingerprints** and compare if they are the same: > gpg --fingerprint

```
student@student-ubuntu:~$ gpg --fingerprint
/home/student/.gnupg/pubring.kbx
--------------------------------
pub   secp256k1 2025-11-16 [SC]
      455C 3D3D FC34 6172 2046  8280 B163 3EFD FF7A 27D1
uid           [ unknown] A_Kali (A_Kali_ECC_key) <AKali@gmail.com>
sub   secp256k1 2025-11-16 [E]

pub   rsa3072 2025-11-16 [SC]
      CA31 4C05 F18E D781 0FEC  EDC0 CFD9 223C A658 52C3
uid           [ unknown] A_Kali (A_Kali_RSA_key) <AKali@gmail.com>
sub   rsa3072 2025-11-16 [E]
```

```
┌──(stud㉿kali-vm)-[~]
└─$ gpg --fingerprint
/home/stud/.gnupg/pubring.kbx
------------------------------
pub   secp256k1 2025-11-16 [SC]
      455C 3D3D FC34 6172 2046  8280 B163 3EFD FF7A 27D1
uid           [ultimate] A_Kali (A_Kali_ECC_key) <AKali@gmail.com>
sub   secp256k1 2025-11-16 [E]

pub   rsa3072 2025-11-16 [SC]
      CA31 4C05 F18E D781 0FEC  EDC0 CFD9 223C A658 52C3
uid           [ultimate] A_Kali (A_Kali_RSA_key) <AKali@gmail.com>
sub   rsa3072 2025-11-16 [E]
```

The fingerprint for A's ECC is: 455C 3D3D FC34 6172 2046 8280 B163 3EFD FF7A 27D1

The fingerprint for A's RSA is: CA31 4C05 F18E D781 0FEC EDC0 CFD9 223C A658 52C3

Machine B needs to create a private key for signing A's keys: > gpg --expert --full-gen-key

```
student@student-ubuntu:~$ gpg --expert --full-gen-key
gpg (GnuPG) 2.4.4; Copyright (C) 2024 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Please select what kind of key you want:
   (1) RSA and RSA
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (7) DSA (set your own capabilities)
   (8) RSA (set your own capabilities)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (sign only)
  (11) ECC (set your own capabilities)
  (13) Existing key
  (14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072)
Requested keysize is 3072 bits
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want for the subkey? (3072)
Requested keysize is 3072 bits
```

```
public and secret key created and signed.

pub   rsa3072 2025-11-16 [SC]
      844E9C8157858469FB551FFBE02AD7EB67C9465C
uid                      B_Ubuntu (B_Ubuntu_RSA_key) <BUbuntu@gmail.com>
sub   rsa3072 2025-11-16 [E]
```

"B_Ubuntu (B_Ubuntu_RSA_key) < BUbuntu@gmail.com>" -> Password: 123456789

To sign the key: A_Kali (A_Kali_ECC_key) <AKali@gmail.com> with the key: B_Ubuntu (B_Ubuntu_RSA_key) BUbuntu@gmail.com

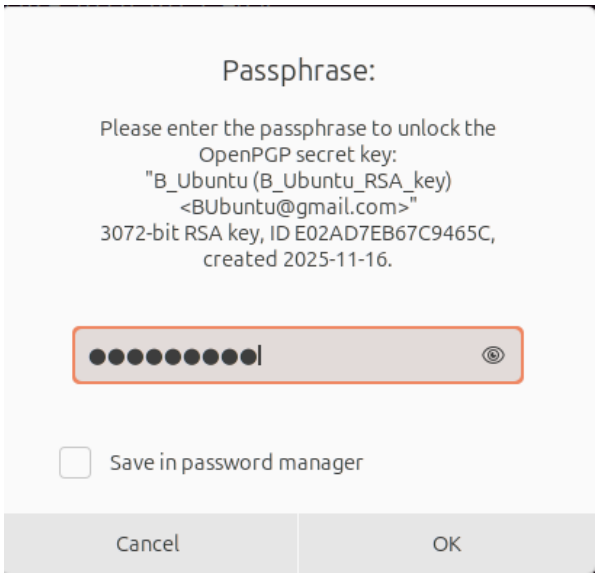
You need to check their keyID by: gpg --list-keys

```
student@student-ubuntu:~$ gpg --list-keys
gpg: checking the trustdb
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: depth: 0  valid:   1  signed:   0  trust: 0-, 0q, 0n, 0m, 0f, 1u
/home/student/.gnupg/pubring.kbx
------------------------------
pub   secp256k1 2025-11-16 [SC]
      455C3D3DFC34617220468280B1633EFDFF7A27D1
uid           [ unknown] A_Kali (A_Kali_ECC_key) <AKali@gmail.com>
sub   secp256k1 2025-11-16 [E]

pub   rsa3072 2025-11-16 [SC]
      CA314C05F18ED7810FECEDC0CFD9223CA65852C3
uid           [ unknown] A_Kali (A_Kali_RSA_key) <AKali@gmail.com>
sub   rsa3072 2025-11-16 [E]

pub   rsa3072 2025-11-16 [SC]
      844E9C8157858469FB551FFBE02AD7EB67C9465C
uid           [ultimate] B_Ubuntu (B_Ubuntu_RSA_key) <BUbuntu@gmail.com>
sub   rsa3072 2025-11-16 [E]
```

You write the command: > gpg --local-user 844E9C8157858469FB551FFBE02AD7EB67C9465C --sign-key 455C3D3DFC34617220468280B1633EFDFF7A27D1

We sign the public keys as we are sure that the keys are fine and have not been altered by anyone.

> gpg --local-user <BKey> --sign-key <Akey>



Once both A keys are signed, check with: > gpg --check-sigs

```
student@student-ubuntu:~$ gpg --check-sigs
gpg: checking the trustdb
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: depth: 0  valid:   1  signed:   2  trust: 0-, 0q, 0n, 0m, 0f, 1u
gpg: depth: 1  valid:   2  signed:   0  trust: 2-, 0q, 0n, 0m, 0f, 0u
/home/student/.gnupg/pubring.kbx
------------------------------
pub   secp256k1 2025-11-16 [SC]
      455C3D3DFC34617220468280B1633EFDFF7A27D1
uid           [  full  ] A_Kali (A_Kali_ECC_key) <AKali@gmail.com>
sig!3        B1633EFDFF7A27D1 2025-11-16  [self-signature]
sig!         E02AD7EB67C9465C 2025-11-16  B_Ubuntu (B_Ubuntu_RSA_key) <BUbuntu@gmail.com>
sub   secp256k1 2025-11-16 [E]
sig!         B1633EFDFF7A27D1 2025-11-16  [self-signature]

pub   rsa3072 2025-11-16 [SC]
      CA314C05F18ED7810FECEDC0CFD9223CA65852C3
uid           [  full  ] A_Kali (A_Kali_RSA_key) <AKali@gmail.com>
sig!3        CFD9223CA65852C3 2025-11-16  [self-signature]
sig!         E02AD7EB67C9465C 2025-11-16  B_Ubuntu (B_Ubuntu_RSA_key) <BUbuntu@gmail.com>
sub   rsa3072 2025-11-16 [E]
sig!         CFD9223CA65852C3 2025-11-16  [self-signature]

pub   rsa3072 2025-11-16 [SC]
      844E9C8157858469FB551FFBE02AD7EB67C9465C
uid           [ultimate] B_Ubuntu (B_Ubuntu_RSA_key) <BUbuntu@gmail.com>
sig!3        E02AD7EB67C9465C 2025-11-16  [self-signature]
sub   rsa3072 2025-11-16 [E]
sig!         E02AD7EB67C9465C 2025-11-16  [self-signature]

gpg: 8 good signatures
```

2.5 Create a text file and sign it using both methods:

Once created the file: > nano importantData.txt

And fill it with important text.

Sign it with OpenSSL: > openssl dgst -sha256 -sign <private_key.pem> -out <signature_file> <input_file>

So: > openssl dgst -sha256 -sign A_OpenSSL_private_key.pem -out importantData.txt.opensslSig importantData.txt

```
┌──(stud㉿kali-vm)-[~]
└─$ openssl dgst -sha256 -sign A_OpenSSL_private_key.pem -out importantData.txt.opensslSig importantData.txt
```

And sign with gpg: > gpg --sign -u <GPG_KEY_ID> --output <signedNameFile> importantData.txt

So: > gpg --sign -u 455C3D3DFC34617220468280B1633EFDFF7A27D1 --output importantData.txt.ECCSig importantData.txt

And: > gpg --sign -u CA314C05F18ED7810FECEDC0CFD9223CA65852C3 --output importantData.txt.RSASig importantData.txt

```
┌──(stud㉿kali-vm)-[~]
└─$ gpg --sign -u 455C3D3DFC34617220468280B1633EFDFF7A27D1 --output importantData.txt.ECCSig importantData.txt

┌──(stud㉿kali-vm)-[~]
└─$ gpg --sign -u CA314C05F18ED7810FECEDC0CFD9223CA65852C3 --output importantData.txt.RSASig importantData.txt
```

2.6 Transfer your file and signatures to another machine/user account and verify the signatures:

After sending the files through sftp as in question 2.3.

To check OpenSSL: > openssl dgst -sha256 -verify <public_key> -signature <signature_file> <data_file>

```
student@student-ubuntu:~$ openssl dgst -sha256 -verify A_OpenSSL_public_key.pem -signature importantData.txt.opensslSig
 importantData.txt
Verified OK
```

To check gpg: gpg --verify <sigFile> / To check the contents: gpg --decrypt <sigFile>

```
student@student-ubuntu:~$ gpg --decrypt importantData.txt.ECCSig
This is data that for any reason must not be changed, so that is why we sign it, so if this data is changed, we can noti
ce.
gpg: Signature made Sun 16 Nov 2025 10:46:15 PM CET
gpg:                using ECDSA key 455C3D3DFC34617220468280B1633EFDFF7A27D1
gpg: Good signature from "A_Kali (A_Kali_ECC_key) <AKali@gmail.com>" [full]
student@student-ubuntu:~$ gpg --decrypt importantData.txt.RSASig
This is data that for any reason must not be changed, so that is why we sign it, so if this data is changed, we can noti
ce.
gpg: Signature made Sun 16 Nov 2025 10:46:28 PM CET
gpg:                using RSA key CA314C05F18ED7810FECEDC0CFD9223CA65852C3
gpg: Good signature from "A_Kali (A_Kali_RSA_key) <AKali@gmail.com>" [full]
```

2.7 After the signature has been successfully verified, modify the content of the original document and verify the digital signature again:

Check OpenSSL modified:

```
student@student-ubuntu:~$ openssl dgst -sha256 -verify A_OpenSSL_public_key.pem -signature importantData.txt.opensslSig
 importantData.txt
Verification failure
40F7E5B0297F0000:error:02000068:rsa routines:ossl_rsa_verify:bad signature:../crypto/rsa/rsa_sign.c:430:
40F7E5B0297F0000:error:1C880004:Provider routines:rsa_verify:RSA lib:../providers/implementations/signature/rsa_sig.c:77
4:
```

Check gpg:

```
student@student-ubuntu:~$ gpg --decrypt EditedimportantData.txt.ECCSig
gpg: no valid OpenPGP data found.
gpg: decrypt_message failed: Unknown system error
student@student-ubuntu:~$ gpg --verify EditedimportantData.txt.ECCSig
gpg: no valid OpenPGP data found.
gpg: the signature could not be verified.
Please remember that the signature file (.sig or .asc)
should be the first file given on the command line.
```

2.8 Only for OpenPGP. Encrypt the file using a public key and then decrypt the encrypted file. Afterward, change one character in the encrypted file and try to decrypt it again:

We encrypt a NormalFile.txt in machine B, and transfer it to machine A:

> gpg --encrypt --recipient keyID --output FileEncryptedName File

```
student@student-ubuntu:~$ gpg --encrypt --recipient 455C3D3DFC34617220468280B1633EFDFF7A27D1 --output EncryptedFile.txt
NormalFile.txt
```

And to decrypt it: > gpg --output DecryptedFile.txt --decrypt EncryptedFile.txt

```
┌──(stud㉿kali-vm)-[~]
└─$ gpg --output DecryptedFile.txt --decrypt EncryptedFile.txt
gpg: encrypted with 256-bit ECDH key, ID CD7AAA52EE47CF56, created 2025-11-16
      "A_Kali (A_Kali_ECC_key) <AKali@gmail.com>"
```

If we change a character in the encrypted file depending on the position we get:

```
┌──(stud㉿kali-vm)-[~]
└─$ gpg --output DecryptedModifiedFile.txt --decrypt EncryptedModifiedFile.txt
gpg: encrypted with 256-bit ECDH key, ID CD7AAA52EE47CF56, created 2025-11-16
      "A_Kali (A_Kali_ECC_key) <AKali@gmail.com>"
gpg: WARNING: encrypted message has been manipulated!
```

```
┌──(stud㉿kali-vm)-[~]
└─$ gpg --output DecryptedModifiedFile1.txt --decrypt EncryptedModifiedFile.txt
gpg: encrypted with 256-bit ECDH key, ID CD7AAA52EE47CF56, created 2025-11-16
      "A_Kali (A_Kali_ECC_key) <AKali@gmail.com>"
gpg: [don't know]: invalid packet (ctb=0d)
gpg: WARNING: encrypted message has been manipulated!
```

So now we are sure if the file is modified or not.

2.9 Is it possible to generate only one of the key pairs for asymmetric algorithms (e.g. private key)? Would this make sense?

Yes, **you can generate just the private key** (the public key is derivable from it), but it's of **limited practical use** unless **you plan to export the public key later** or keep the private key locked in an HSM. Never share private keys.

2.10 What form does the GPG key have? How does a PEM key differ from an Open-PGP key?

GPG / OpenPGP: keys are OpenPGP packets (key material + user-IDs + signatures/subkeys), often ASCII-armored (-----BEGIN PGP PUBLIC KEY BLOCK-----).
PEM: Base64/PEM-wrapped ASN.1/DER structures (PKCS#1/PKCS#8/X.509) holding a single key or certificate.
Main differences: internal format/metadata, trust model (web-of-trust vs PKI), and fingerprint calculation.

2.11 Is the exchange of private keys justified?

Generally, no, **private keys must remain secret**. Only justified for secure, audited actions (key migration/backup, escrow, or threshold schemes) using encrypted channels, HSMs, or split-secret techniques.

2.12 What is the fingerprint?

A fingerprint is a **short cryptographic hash of a key's canonical** representation used as a stable, **human-checkable identifier** to detect swapped or altered keys.

The fingerprint is calculated by hashing 0x99 | 2 bytes of the public key length | public key.

2.13 Describe what has changed and why in task 7?

You **changed the signed document's bytes**; because a signature covers the exact data, verification now fails **so integrity is broken**.

2.14 Did you succeed in changing one character in an encrypted file and decrypt the encrypted text? Justify your answer. (Task 8):

No, **altering one character in ciphertext will** (with modern OpenPGP's MDC/AEAD) **cause decryption to fail** or the integrity check to trigger; you won't get the original plaintext.

2.15 What is the difference in the signature (OpenPGP) for ECC and RSA algorithms?

RSA signs with modular-exponent math (large integers); **ECC uses elliptic-curve schemes** (ECDSA/EdDSA). ECC gives smaller keys/signatures and comparable security; OpenPGP stores the algorithm ID, but user-level verification is the same.

2.16 Can a message be signed with a public key? If so, what could the consequences be?

No, signing requires the private key. "Signing with a public key" is impossible in any meaningful sense; if it were possible the signature would prove nothing (anyone could create it), so **authenticity would be lost**.

3 PKI - key generation in X.509 standard -> Build CA:

3.1 Change the vars after cp them: > nano vars

```
#set_var EASYRSA_DN      cn_only

# Organizational fields (used with "org" mode and ignored in "cn_only" mode).
# These are the default values for fields which will be placed in the
# certificate.  Do not leave any of these fields blank, although interactively
# you may omit any specific field by typing the "." symbol (not valid for
# email).
#
# NOTE: The following characters are not supported
#       in these "Organizational fields" by Easy-RSA:
#       back-tick (`)
#
set_var EASYRSA_REQ_COUNTRY     "PL"
set_var EASYRSA_REQ_PROVINCE    "Wroclaw"
set_var EASYRSA_REQ_CITY        "Wroclaw"
set_var EASYRSA_REQ_ORG         "PWR"
set_var EASYRSA_REQ_EMAIL       "alvaroscar@gmail.com"
set_var EASYRSA_REQ_OU          "PWR"

# Preserve the Distinguished Name field order
# of the certificate signing request
# *Only* effective in --dn-mode=org
#
#set_var EASYRSA_PRESERVE_DN    1

# Set no password mode - This will create the entire PKI without passwords.
# This can be better managed by choosing which entity private keys should be
# encrypted with the following command line options:
# Global option '--no-pass' or command option 'nopass'.
#
#set_var EASYRSA_NO_PASS        1
```

3.2 Generation of the keys: > ./easyrsa build-ca nopass

4 Generate the server and client certificates:

4.1 Run the command to generate a server private key (server.key) and a certificate signing request - CSR (server.req):

./easyrsa gen-req server nopass

4.2 Sign a server request using CA, it generates server.crt:

./easyrsa sign-req server server

4.3 Run the command to generate a client private key (client1.key) and a certificate signing request (client1.req):

./easyrsa gen-req client1 nopass:

4.4 Sign a client signing request using CA, it generates client1.crt:

./easyrsa sign-req client client1

```
┌──(stud㉿kali-vm)-[~/…/VPN/server/easy-rsa/easyrsa3]
└─$ ./easyrsa sign-req client client1
Using Easy-RSA 'vars' configuration:
* /home/stud/Desktop/VPN/server/easy-rsa/easyrsa3/vars
Please check over the details shown below for accuracy. Note that this request
has not been cryptographically verified. Please be sure it came from a trusted
source or that you have verified the request checksum with the sender.
You are about to sign the following certificate:

  Requested CN:     'client1'
  Requested type:   'client'
  Valid for:        '825' days


subject=
    commonName                = client1

Type the word 'yes' to continue, or any other input to abort.
  Confirm requested details: yes

Using configuration from /home/stud/Desktop/VPN/server/easy-rsa/easyrsa3/pki/45201999/temp.00
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
commonName            :ASN.1 12:'client1'
Certificate is to be certified until Feb 21 18:29:23 2028 GMT (825 days)

Write out database with 1 new entries
Database updated

WARNING
═══════
INCOMPLETE Inline file created:
* /home/stud/Desktop/VPN/server/easy-rsa/easyrsa3/pki/inline/private/client1.inline


Notice
──────
Certificate created at:
* /home/stud/Desktop/VPN/server/easy-rsa/easyrsa3/pki/issued/client1.crt
```

4.5 Generate Diffie-Hellman key (dh.pem) which is used to exchange keys:

./easyrsa gen-dh

4.6 Generate an HMAC signature (ta.key) to strengthen the server's TLS integrity verification capabilities:

openvpn --genkey --secret ta.key

4.7 What is the signature algorithm used?

The signature algorithm used in the certificate is **sha256 with RSA Encryption**, which means the certificate was signed using RSA and the SHA-256 hashing algorithm.

4.8 What is the public key algorithm used?

The public key algorithm used is **RSA Encryption (typically with a 2048-bit RSA key).**

4.9 What version of the X.509 standard has been used to generate a certificate?

Certificates generated by EasyRSA and OpenSSL use X.509 version 3.
This is the most modern version, and it allows the inclusion of extensions such as SAN, Key Usage, etc.

4.10 What else can be read from the server/client certificate?

X.509 certificates contain much more information than just the **public key**. In addition to the signature and public key algorithms, a certificate also includes the **issuer** (the CA that signed it), the **subject** (identity of the server or client), the **validity period**, the **serial number**, **fingerprints**, and several X.509 v3 extensions such as Key Usage, Extended Key Usage, Basic Constraints, and identifiers for both the subject and the issuer. These fields define how the certificate can be used and help establish trust in a PKI system.